

Lincoln University College
Faculty of Computer Science and Multimedia

**PocketDRS: A Single-View 3D Trajectory
Reconstruction and Decision Review System for Cricket**

MID DEFENSE REPORT

Submitted to
Department of Information Technology
Phoenix College of Management
Maitidevi, Kathmandu

In partial fulfillment of the requirements for the degree of
Bachelor in Information Technology (BIT)

Submitted by
Niraj Kafle
BIT 7th Semester
University ID: LC0003001674

March, 2026

Contents

1	Introduction	1
2	Problem Statement	1
3	Objectives	1
3.1	General Objective	1
3.2	Specific Objectives	2
4	Methodology	2
4.1	Requirement Identification	2
4.1.1	Study of Existing System	2
4.1.2	Literature Review	2
4.1.3	Requirement Analysis	2
4.2	Feasibility Study	3
4.2.1	Technical Feasibility	3
4.2.2	Operational Feasibility	3
4.2.3	Economic Feasibility	5
4.3	High Level Design of System	5
4.3.1	Context Level DFD	5
4.3.2	Level 1 DFD	5
4.3.3	Working Flow	5
4.3.4	System Components	5
4.3.5	Deployment View	5
4.3.6	Current Implementation Status	5
5	Gantt Chart	10
6	Expected Outcome	10
7	References	11

1 Introduction

Cricket uses more technology every year. In top level matches, systems like Hawk-Eye help umpires review close decisions. These systems work well, but they need many cameras, special equipment, and trained operators. That makes them too expensive for local grounds, schools, clubs, and small tournaments [1].

PocketDRS is built to solve that gap. It tries to give a simple decision support system using a single phone camera and a lightweight processing server. The user records a delivery, marks pitch points on the screen, submits the clip, and gets a result with a ball path view and an LBW decision.

The project focuses on a practical version of DRS that people can actually use in low-cost cricket setups. Instead of chasing perfect broadcast level accuracy, the goal is to produce a stable and understandable result from normal hardware.

2 Problem Statement

Professional DRS systems depend on multiple synchronized cameras. They find the ball in different views and rebuild its path in 3D. That approach is accurate, but it is expensive and hard to use outside major venues.

When the same problem is moved to a single camera, three main issues appear:

1. A single camera does not give direct depth information.
2. A cricket ball is small, fast, and easy to lose in the frame.
3. Any small calibration mistake can affect the final decision.

The main question of this project is simple: can one phone camera and a lightweight vision pipeline track the ball well enough to help with LBW review on ordinary cricket grounds?

3 Objectives

3.1 General Objective

To build a simple and low-cost cricket review system that can track the ball from a single camera view, estimate its path, and support LBW decisions.

3.2 Specific Objectives

1. Study existing DRS tools and similar research to identify methods that can work on normal devices.
2. Build a pitch calibration step that maps screen points to real pitch positions.
3. Track the ball using motion and colour based detection.
4. Detect bounce and impact points from the tracked path.
5. Estimate whether the ball would hit the stumps.
6. Show the result in a clear 3D view that is easy to understand.

4 Methodology

4.1 Requirement Identification

4.1.1 Study of Existing System

Existing systems such as Hawk-Eye use many cameras, shared calibration data, and powerful servers. They are designed for large professional setups. PocketDRS uses a smaller workflow. It keeps one camera, one trimmed clip, a user guided calibration step, and a simpler analysis pipeline. This makes the system cheaper and easier to deploy.

4.1.2 Literature Review

Related work shows that single-view sports tracking becomes possible when the scene has known geometry and the motion follows a predictable pattern. In cricket, the pitch gives fixed reference points, and the ball usually follows a smooth path between release, bounce, impact, and wicket line. Homography-based mapping from a calibrated view can project 2D detections onto a known plane [3]. Background subtraction methods like MOG2, combined with colour-space filtering, are effective for real-time ball detection in controlled settings [4]. Restricting the detection region to the pitch area further reduces false positives from players and the surrounding environment. This makes a practical single-view solution possible, even if it is simpler than a full professional DRS [2].

4.1.3 Requirement Analysis

Functional requirements

1. User signs in.
2. User records or selects a video.
3. User creates or reuses pitch calibration.
4. User trims the clip and submits it.
5. Server tracks the ball.
6. Server maps the path to the pitch.
7. Server detects bounce and impact.
8. Server checks LBW outcome.
9. App shows the result in a simple 3D view.

Non-functional requirements

1. The system should stay easy to use for a first-time user.
2. Processing should finish in a short time for one delivery.
3. The app should run on both Android and iOS.
4. The server should work with normal video quality and common frame rates.
5. The final result should be clear enough for a user to follow without technical knowledge.

4.2 Feasibility Study

4.2.1 Technical Feasibility

The current prototype uses tools that are already stable and widely used. Flutter handles the mobile app. FastAPI handles the server API. OpenCV handles video analysis. Three.js is used for the 3D result view inside a WebView. This stack is realistic for a student project and works on consumer hardware.

4.2.2 Operational Feasibility

The workflow is simple enough for normal use. A phone on a tripod records the ball. The user marks pitch points once, trims the clip, and waits for the result. The steps are short and easy to repeat during testing.

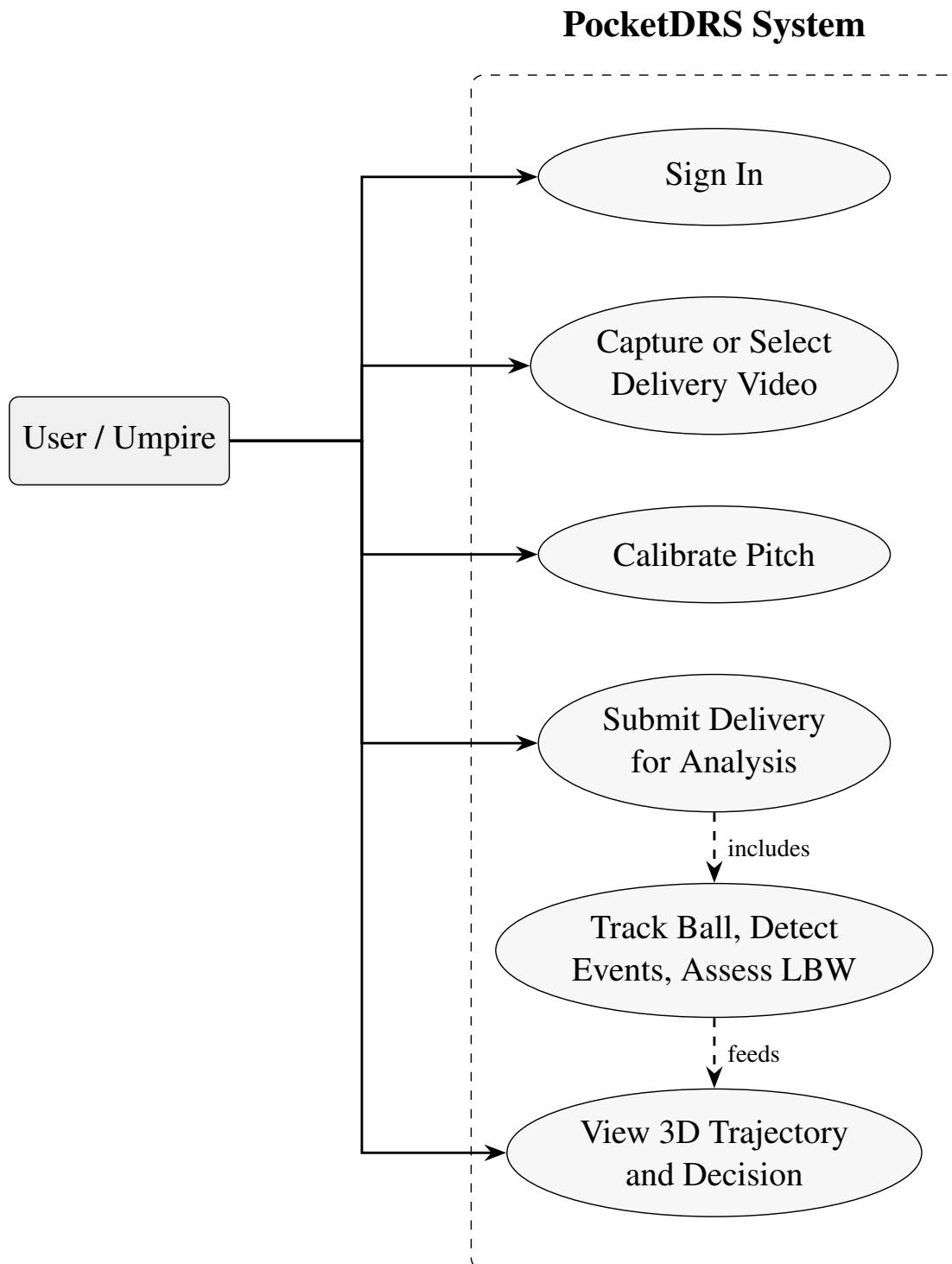


Figure 1: Main use cases of PocketDRS.

4.2.3 Economic Feasibility

The project does not need special sensors or paid software. It uses open source libraries and devices that are already available. That keeps the cost very low.

Table 1: Main tools used in PocketDRS.

Part	Tool
Mobile app	Flutter and Dart
Server API	FastAPI and Python
Vision pipeline	OpenCV, NumPy, SciPy
Authentication	Firebase Authentication
Data storage	Cloud Firestore
3D result view	Three.js inside WebView
Testing	pytest and Flutter analyzer

4.3 High Level Design of System

The design can be understood through the following views: overall system architecture, context-level data flow, level-1 processing flow, application activity flow, software components, and deployment structure. Together these diagrams show how PocketDRS moves from user input to final LBW decision output.

4.3.1 Context Level DFD

4.3.2 Level 1 DFD

4.3.3 Working Flow

The working flow is kept short so that the user can follow it easily:

4.3.4 System Components

4.3.5 Deployment View

4.3.6 Current Implementation Status

The project is not only planned on paper. A working prototype already exists and has gone through a full cycle of implementation and improvement. Below is the current state of each part of the system.

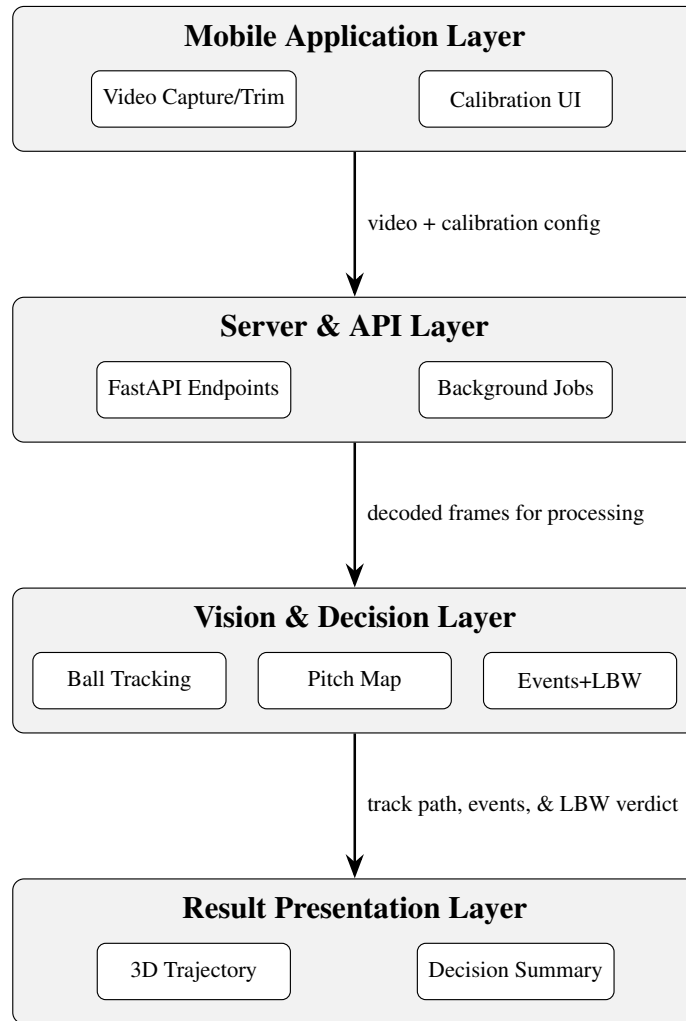


Figure 2: Overall architecture of PocketDRS.

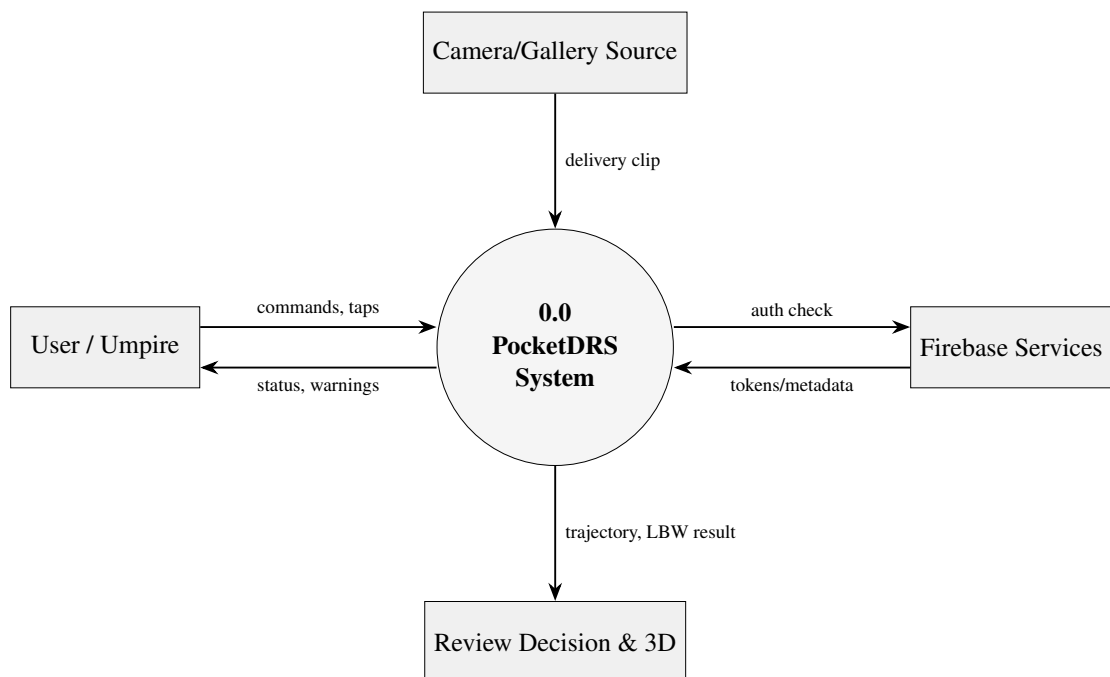


Figure 3: Context-level DFD of PocketDRS.

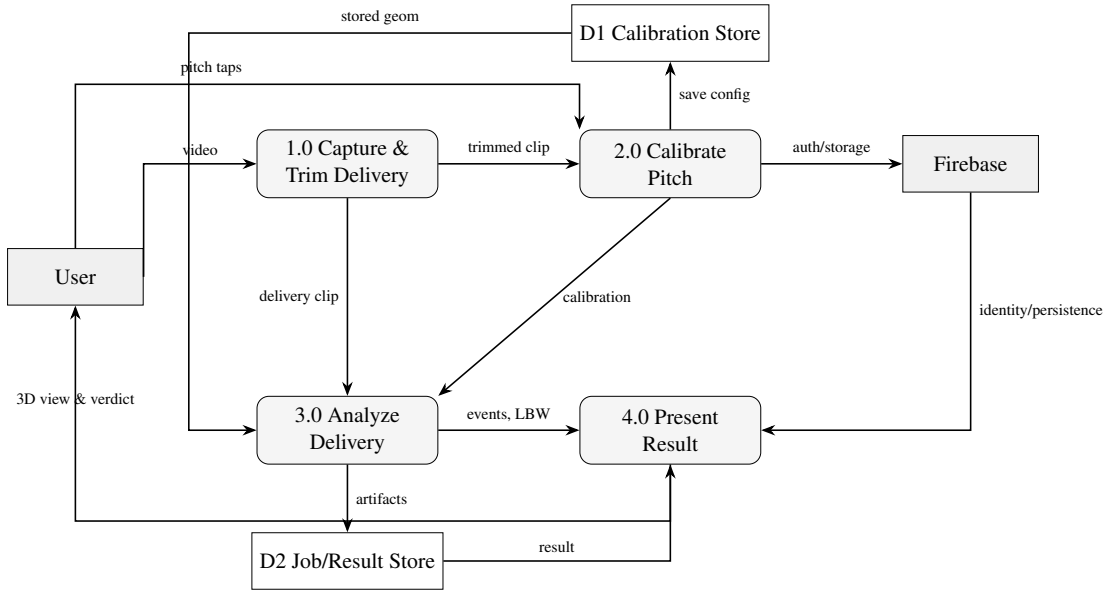


Figure 4: Level-1 DFD of the PocketDRS processing workflow.

Mobile Application. The Flutter app supports sign in with Firebase, video selection and recording, an interactive pitch calibration step where the user taps four pitch corners and two stump bases, video trimming, submission to the server, live progress polling, and a full result screen with a 3D trajectory view. The calibration screen validates user inputs and checks for geometric consistency before allowing the user to proceed.

Server and API. The FastAPI server accepts video uploads and processes them as background jobs. Each job goes through frame decoding, ball tracking, calibration mapping, bounce and impact detection, and LBW assessment. The job API returns structured results including track points, pitch plane coordinates, event indices, and an LBW decision with reasons.

Ball Tracking Pipeline. Tracking uses a combined motion and colour detection approach. A MOG2 background subtractor detects moving objects while an HSV colour detector finds the ball by its known colour. The two detectors are fused so that matches between them receive higher confidence. A recent improvement adds a region-of-interest (ROI) mask built from the calibration corners, which restricts detection to the pitch area and its surroundings. This eliminates false positives from fielders, boundary boards, and camera shake in the background.

Pitch Calibration and Homography. The user marks four pitch corners and two stump bases on a reference frame. The server uses these to compute a homography matrix that maps pixel positions to real pitch coordinates. Stump base points refine the homography. A reprojection quality score warns the user when calibration may be inaccurate.

Event Detection and LBW. Bounce and impact events are detected using both pixel-space and pitch-plane heuristics. The system looks for velocity changes and lateral direction shifts

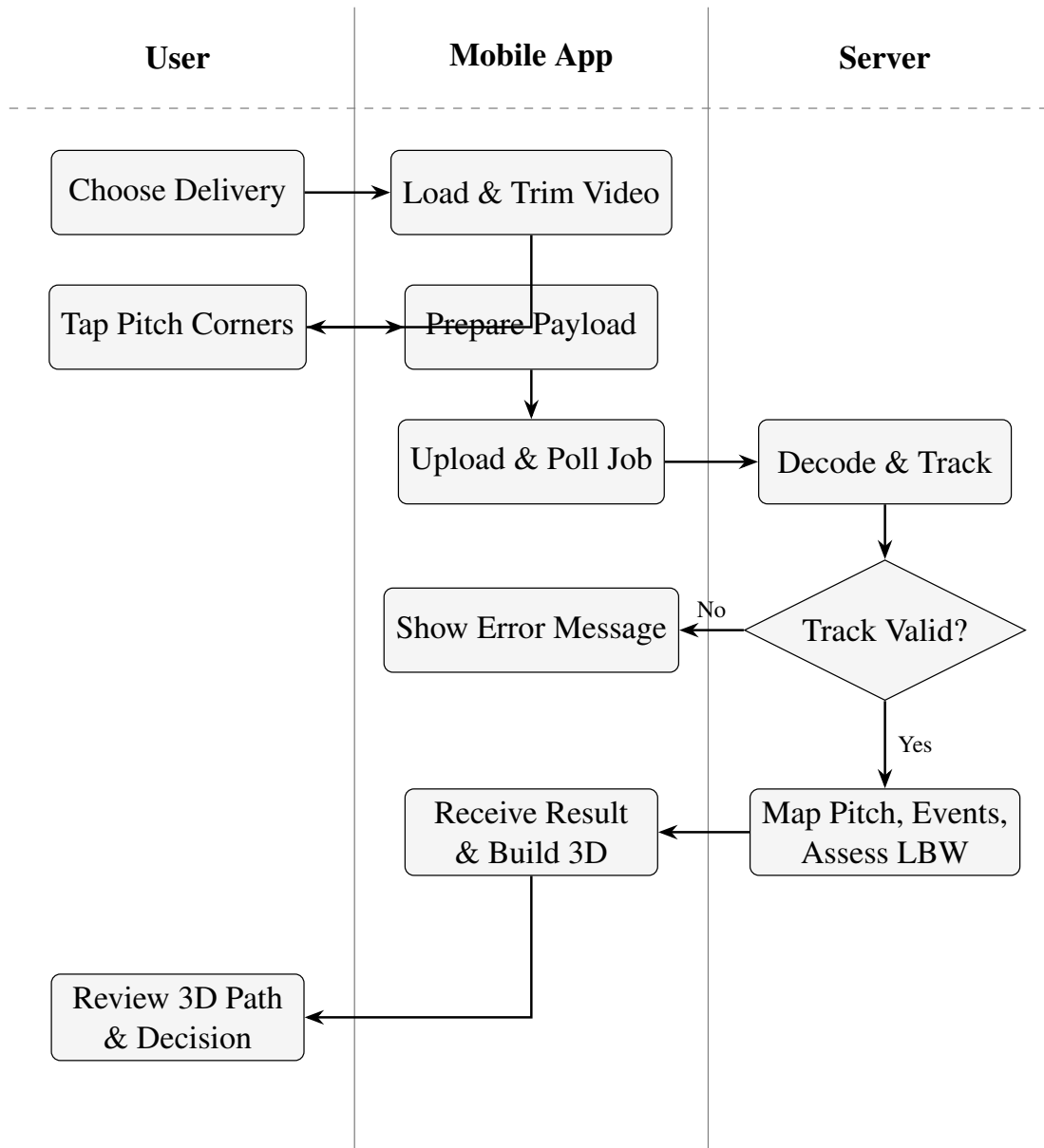


Figure 5: Main processing flow used by the system.

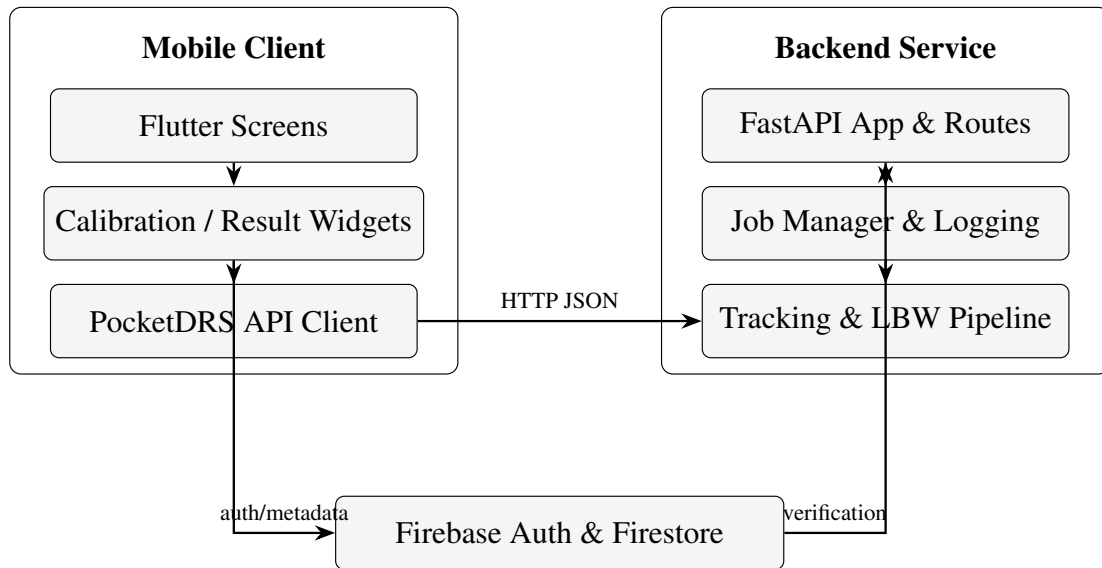


Figure 6: Main software components.

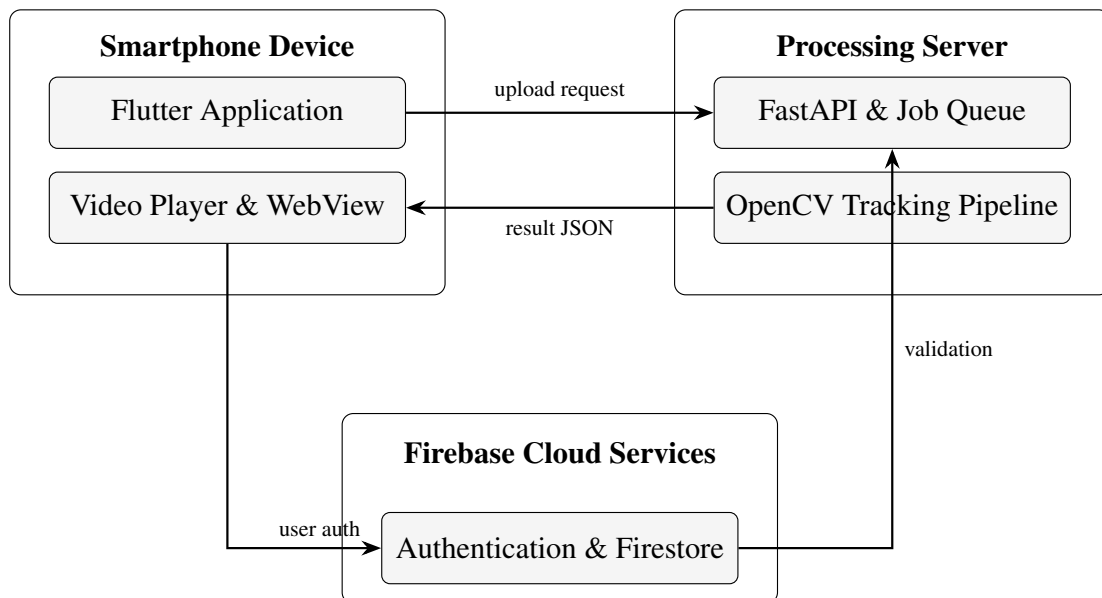


Figure 7: Deployment view of the working system.

to identify the bounce point, and finds the first point near the striker crease as impact. The LBW module fits a curve through the post-bounce trajectory and evaluates where the ball would cross the stump line. It checks pitching in line, impact in line, and whether the ball would hit the stumps, and reports a decision with a confidence metric.

3D Visualization. The result view uses Three.js loaded in a WebView. It renders a realistic cricket pitch with stumps, crease lines, and the ball trajectory as coloured tubes (blue before bounce, green after bounce, amber for the predicted path). Bounce and impact points are shown as highlighted markers. The user can rotate the view using touch controls.

5 Gantt Chart

Table 2: Project schedule.

Task	Dec	Jan	Feb	Mar	Apr
Problem study and review	X	X			
System design		X	X		
App and server development		X	X	X	
Tracking and decision logic			X	X	
Testing and refinement				X	X
Final improvement and report work				X	X

6 Expected Outcome

The expected outcome of PocketDRS is a working low-cost review tool for cricket that can:

1. Accept a single delivery clip from a phone.
2. Map the video to the cricket pitch using user selected points.
3. Track the ball through the important part of the delivery.
4. Mark bounce and impact in a useful way.
5. Give a simple LBW decision with a clear visual result.
6. Help clubs and student level cricket use a DRS-like tool without expensive equipment.

If accuracy improves further with more testing data, the same system can later grow into a stronger tool for coaching, match review, and training support.

7 References

References

- [1] Hawk-Eye Innovations, “Hawk-Eye: The Technology Behind Sports Officiating,” 2024. [Online]. Available: <https://www.hawkeyeinnovations.com/>
- [2] P. Ponglertnapakorn, T. Shirai, and T. Yamasaki, “Where Is the Ball: 3D Ball Trajectory Estimation From 2D Monocular Tracking,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021.
- [3] Z. Zhang, “A Flexible New Technique for Camera Calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000.
- [4] OpenCV Documentation, “Camera Calibration and 3D Reconstruction,” 2024. [Online]. Available: <https://docs.opencv.org/>
- [5] International Cricket Council, “Playing Conditions: Law 36 Leg Before Wicket,” 2024.
- [6] Google, “Flutter: Build apps for any screen,” 2024. [Online]. Available: <https://flutter.dev/>
- [7] S. Ramírez, “FastAPI: Modern, fast web framework for building APIs with Python,” 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
- [8] Three.js Contributors, “Three.js: JavaScript 3D Library,” 2024. [Online]. Available: <https://threejs.org/>